Practice midterm 3 (with solutions)

TC1037 Algorithmic Thinking · Group 510 · Tecnológico de Monterrey, Campus Querétaro

Practice — download, solve on paper in 80 minutes, then check the solutions

Name: _____ **Matrícula:** _____

80 minutes · 100 points · individual · on paper. No computer, phone, notes or calculator. Write in pen or dark pencil; code must be legible and indented. **Part I** (30 pts): write *exactly* what Python prints — every line, in order. **Part II** (70 pts): write the program. Suggested time: 15 minutes for Part I, 65 for Part II. **This is a practice paper: same format, same difficulty, different problems.**

Question:	1	2	3	4	5	6	7	8	9	10	11	Total
Points:	5	5	5	5	5	5	23	23	3	20	1	100
Score:												

Part I — Read and predict

(30 points)

1. **Arithmetic operators.** What does Python print? [5]

```
print(3 * 4 % 5 + 17 // 2 ** 2 - 1)
```

Solution:
```
5
```

2. **Boolean operators.** What does Python print? [5]

```
print(not 5 % 2 == 0 and 3 <= 3 or 10 / 4 == 2)
```

Solution:
```
True
```

3. **Conditional structure.** What does Python print? [5]

```
x = 15
y = 4
if x > 10:
    if x % y == 0:
        print("A", x // y)
    else:
        print("B", x % y)
else:
    print("C")
print("end")
```

Solution:
```
B 3
end
```

4. **Functions.** What does Python print? [5]

```
def double_minus(a, b):
    a = a * 2
    return a - b

def main():
    a = 4
    b = 1
    b = double_minus(b, a)
    a = double_minus(a, b)
    print(a, b)

main()
```

Solution: `10 -2`

5. **The while loop.** What does Python print? [5]

```
n = 6
steps = 0
while n != 1:
    if n % 2 == 0:
        n = n // 2
    else:
        n = 3 * n + 1
    steps = steps + 1
print(steps)
```

Solution: `8`

6. **The for loop.** What does Python print? [5]

```
count = 0
for i in range(10, 0, -3):
    if i % 2 == 0:
        count = count + i
    print(i, count)
```

Solution: `10 10`
`7 10`
`4 14`
`1 14`

Part II — Write the program **(70 points)**

Write **one** Python program with the following parts. All `input()` and all validation happen in `main()`; the two functions **return** their result and never print or read. Write each function's purpose in a one-line comment above it, and under each loop a one-line comment saying why it stops (`while`) or how many times it runs (`for`).

7. The function `largest_digit(n)` receives a non-negative integer `n` and **returns** its largest digit. Use a `while` loop with `%` and `//`; do not convert `n` to a string. [23]

   Examples: `largest_digit(3917)` → `9` `largest_digit(5)` → `5` `largest_digit(1000)` → `1` `largest_digit(0)` → `0`

8. The function `factorial(n)` receives an integer $n \geq 0$ and **returns** $n! = 1 \cdot 2 \cdot 3 \cdots n$ (by definition, $0! = 1$). Use a `for` loop; do not use `math.factorial`. [23]

   Examples: `factorial(0)` → `1` `factorial(1)` → `1` `factorial(4)` → `24` `factorial(6)` → `720`

9. The function `menu()` that displays: `1. largest_digit  2. factorial  3. Exit` [3]

10. The function `main()` that, inside a `while` loop: shows the menu, reads the option, and [20]
    - option 1: reads n (must be 0 or more); if any value is out of range displays a clear message, otherwise calls `largest_digit` and displays the result with a label;
    - option 2: reads n (must be 0 or more); same rule, calling `factorial`;
    - option 3: displays `Goodbye.` and the program ends;
    - any other option: displays `Invalid option.` and the program ends.

    Use `if`/`elif`/`else` to choose the option.

11. The last line of the script calls `main()`. [1]

Technical requirements — 2 points off for each one missing: meaningful variable names in Python style · a clear prompt for every input · readable, labelled output.

Reference solution

```python
# Reference solution — Practice midterm 3 (with solutions)

def largest_digit(n):
    largest = 0
    while n > 0:
        digit = n % 10
        if digit > largest:
            largest = digit
        n = n // 10
    return largest


def factorial(n):
    product = 1
    for i in range(1, n + 1):
        product = product * i
    return product
```

```
def menu():
    print("1. largest_digit")
    print("2. factorial")
    print("3. Exit")


def main():
    keep_going = True
    while keep_going:
        menu()
        option = int(input("Option: "))
        if option == 1:
            n = int(input("Number n (0 or more): "))
            if n < 0:
                print("Invalid input: n must be 0 or more.")
            else:
                print("Largest digit:", largest_digit(n))
        elif option == 2:
            n = int(input("Number n (0 or more): "))
            if n < 0:
                print("Invalid input: n must be 0 or more.")
            else:
                print("Factorial:", factorial(n))
        elif option == 3:
            print("Goodbye.")
            keep_going = False
        else:
            print("Invalid option.")
            keep_going = False


main()
```

How Part II is graded (the academia's rubric)

Each part of the program (the two functions, `menu`, `main`, the call) is scored on this scale, as a percentage of its points:

Excelente 100%	**Muy bien 85%**	**Regular 70%**	**Deficiente 50%**	**Muy deficiente 20%**	**Sin evidencia 0%**
No syntax errors; works correctly, including the boundary cases in the examples.	Correct approach; minimal logic or syntax slips.	At most two syntax errors and/or small logic errors; the approach is closely related to what was asked.	Approach loosely related, or related but with more than two syntax errors.	Code unrelated to the problem.	No code.

Part I: 5 points per question, all or nothing — every line of the output, in order. Technical requirements: -2 each, from Part II.